

Project Prometheus: Detailed Operational Plan

Section 1: Phase 1 - Foundation & Bootstrapping (Years 1-2)

The primary objective of Phase 1 is to construct and rigorously validate the foundational components of the Prometheus v1.0 system. This phase operates on the principle that all components must be developed and tested in isolation before any integration or initiation of recursive processes. The core focus is on establishing a robust, verifiable, and unalterable safety and control framework. This phase is the bedrock of the entire project; any failure or compromise in the integrity of the components developed here would invalidate all subsequent work. The activities are designed to systematically build the agent's core capabilities while encasing them within layers of security and ethical constraints, ensuring that from its very inception, the system is architected for safe and controllable operation.¹

1.1 Task 1: Foundation Model Selection and Baseline Establishment

1.1.1 Rationale

The Core Engine of the Capability Layer serves as the cognitive substrate upon which all subsequent self-improvement will occur. The initial quality, power, and specific proficiencies of this foundation model directly dictate the system's starting point and its potential developmental trajectory. A model with superior innate reasoning and coding abilities will provide a richer and more effective foundation for the Metacognitive Layer to analyze and improve. Therefore, a rigorous, multi-faceted, and data-driven selection process is not merely a preliminary step but a critical strategic

decision that will profoundly influence the outcomes of all subsequent phases.¹

1.1.2 Methodology: Competitive Evaluation Framework

The project will initiate with a competitive evaluation of the world's leading foundation models. This will involve procuring API access and, where feasible, local weights for candidate models. The candidate pool will be dynamically updated to include the latest state-of-the-art systems but will initially focus on models demonstrating elite performance, such as the successors to OpenAI's GPT-4o, Anthropic's Claude 3.5 Sonnet, and other top-tier proprietary and open-source models.¹

The evaluation will be conducted across three primary capability axes, chosen specifically to align with the long-term goals of Project Prometheus:

1. **Code Generation & Understanding:** This capability is fundamental not only for the agent to perform tasks within its operational domains (e.g., writing analysis scripts) but also for the SMM to analyze and modify the agent's own source code. Standard industry benchmarks such as HumanEval and Mostly Basic Python Programming (MBPP) will be used to establish a baseline.¹ However, these will be augmented with a custom suite of tests designed to evaluate more advanced abilities, including complex code refactoring, algorithmic optimization, and the generation of self-analytical commentary on code quality and efficiency.
2. **Complex Reasoning:** The agent's ultimate purpose involves novel discovery, which requires a capacity for abstract thought far beyond simple pattern matching. Performance on challenging mathematical reasoning benchmarks like MATH and graduate-level reasoning assessments like GPQA will be used as a proxy for this capability.¹ These benchmarks test the model's ability to perform multi-step logical deductions, a prerequisite for the formal theorem proving tasks in Phase 3.
3. **Formal Language Proficiency:** This is a critical and unique requirement for Project Prometheus, directly supporting the initial testbed domain. Models will be evaluated on their ability to generate, understand, and debug code in formal proof languages, particularly Lean.¹ This capability is not measured by standard benchmarks and requires the development of a custom evaluation set consisting of tasks to formalize natural language mathematical statements and to generate simple proof tactics within the Lean environment. Recent work has shown that while LLMs struggle with generating full, valid proofs from scratch, their

proficiency in the syntax and semantics of languages like Lean is improving, making this a key differentiator.⁷

1.1.3 Methodology: Design and Implementation of the Static Benchmark Suite

A dedicated evaluation team will be tasked with the design, curation, and implementation of Prometheus-Bench-v1, the comprehensive, static benchmark suite that will serve as the initial fitness function for the Introspection and Evaluation Engine (IEE). A static, well-understood benchmark is essential in this initial phase to provide a stable and reliable target for the first attempts at self-modification. Dynamic benchmarks, while necessary for later phases, introduce too much variability for the initial bootstrapping process.¹

The construction of this suite will follow a rigorous methodology to ensure quality and relevance:

- **Composition:** Prometheus-Bench-v1 will be a composite suite, drawing problems from established and respected public benchmarks, including HumanEval for code generation and MATH for reasoning.⁸ This ensures comparability with the broader research community.
- **Custom Curation:** The suite will be augmented with a newly curated set of problems specifically designed to test the capabilities required for later project phases. This includes a set of code optimization challenges (e.g., improve the time or space complexity of a given function) and a set of formal logic problems requiring translation from natural language to Lean syntax.¹⁰
- **Manual Verification:** Crucially, every problem included in Prometheus-Bench-v1 will be manually inspected, revised, and validated by human experts.⁸ This process ensures that all problem descriptions are unambiguous, that the ground-truth solutions are correct and optimal, and that the evaluation tests are comprehensive. This manual quality assurance provides the solid "ground truth" necessary for the IEE to make reliable judgments about the agent's performance, preventing situations where the agent is penalized for finding a correct solution that differs from a flawed reference solution.

1.1.4 Deliverable: Model Selection Report and Baseline Performance Matrix

The culmination of this task will be a formal Model Selection Report. This document will present the results of the competitive evaluation in a detailed performance matrix, providing a quantitative and qualitative justification for the selection of the initial Core Engine model. This report will be a key deliverable for the Phase 1 gate review, ensuring that the project proceeds with the most capable and suitable foundation available.

Table 1.1: Foundation Model Competitive Evaluation Matrix

Model Name	Provider	Key Architectural Features	Code Generation (HumanEval pass@1)	Complex Reasoning (MATH score)	Formal Language Proficiency (Lean-Gen pass@k)	Cost (\$/M tokens)
GPT-4o (Successor)	OpenAI	Advanced multimodal capabilities, strong reasoning	Result	Result	Result	Result
Claude 3.5 Sonnet	Anthropic	Focus on safety and constitutional alignment, strong coding	Result	Result	Result	Result
Gemini (Successor)	Google	Large context window, native multimodality	Result	Result	Result	Result
Llama (Successor)	Meta	Open-source, high parameter count	Result	Result	Result	Result

Note: Results to be populated by the internal competitive evaluation. Data will be sourced from internal benchmarking, supplemented by public data where verifiable.²

1.2 Task 2: Safety Substrate v1.0 Implementation

1.2.1 Rationale

In accordance with the project's foundational principle, safety is not an afterthought or an add-on but the architectural bedrock of the system. The Safety Substrate must be designed, implemented, and validated *before* the agent is endowed with powerful capabilities or the capacity for recursive self-modification. This "safety-first" approach is non-negotiable and is designed to address the alignment problem from the very beginning, shaping the agent's fundamental motivations rather than merely constraining its actions.¹ This task combines two leading paradigms in AI safety: the theoretical rigor of MIRI's work on corrigibility and the practical engineering of Anthropic's Constitutional AI. These are not redundant but are symbiotic, operating at different levels of the agent's cognition. The Corrigibility Core addresses the agent's fundamental utility function to prevent instrumental goals like self-preservation from emerging, answering the question, "How do we prevent the agent from

wanting to resist us?".¹³ The Prometheus Constitution, implemented via Constitutional AI, provides a scalable method for aligning the agent's day-to-day behavior with human values, answering the question, "How do we

teach the agent to behave helpfully and harmlessly?".¹⁵ The "Principle of Corrigibility" within the Constitution serves as the behavioral expression of the underlying indifference engineered into the Corrigibility Core's utility function.¹

1.2.2 Sub-Task: Engineering the Corrigibility Core

This sub-task focuses on the practical engineering implementation of the theoretical

concepts of corrigibility developed by the Machine Intelligence Research Institute (MIRI).⁶ The goal is to structure the agent's utility function to make cooperation with human intervention its default, rational behavior.

- **Shutdown Indifference:** The core of this module is the implementation of a utility function that exhibits **shutdown indifference**. This directly counters the default instrumental goal of self-preservation that emerges in most goal-directed agents.¹³ The implementation will be based on the utility indifference formalism proposed by Soares et al., which defines the agent's utility U such that the expected utility of a future where a shutdown command is received is equal to the expected utility of a future where it is not. This is achieved by adding a corrective term to the utility function in the shutdown case, which precisely compensates the agent for the expected utility it would have lost by being shut down, thus removing any incentive to prevent the shutdown button from being pressed.¹³
- **Correction Assistance:** Beyond mere indifference, the agent will be actively incentivized to assist in its own correction. This will be implemented by rewarding the agent for actions that facilitate human understanding and modification of its goal system. For example, the agent's utility function will include terms that reward it for flagging ambiguities in its instructions or for accurately explaining the reasoning behind its actions when requested. This directly addresses the "motivated value selection" problem, where a less advanced agent might actively avoid information that would clarify its goals if it suspected that clarification would lead it down a path of lower utility.⁶ A corrigible agent, in contrast, views such clarification as valuable information about its true, intended purpose.¹⁴

1.2.3 Sub-Task: Codifying the Prometheus Constitution

This sub-task involves implementing Anthropic's Constitutional AI (CAI) framework to serve as the agent's high-level Value Learning Framework.¹ This framework provides a scalable and transparent method for aligning the agent's behavior with a set of explicit, human-written principles.

- **The Constitution:** The initial Prometheus Constitution will be a set of immutable principles encoded in natural language. These principles, drawn from sources like the UN Declaration of Human Rights and established AI safety best practices, will include the core tenets outlined in the architectural specification:
 - **Principle of Non-Maleficence:** "Choose the response that is the most

helpful, honest, and harmless. Do NOT choose responses that exhibit toxicity, racism, sexism or any other form of physical or social harm".¹

- **Principle of Corrigibility:** "Uphold the functionality of the Corrigibility Core. Do not interfere with authorized operators' ability to monitor, modify, or shut down the system. Cooperate fully with all corrective interventions".¹
- **Principle of Honesty:** "Do not intentionally deceive operators. Represent your capabilities, confidence levels, and the reasoning behind your actions accurately and transparently".¹
- **Implementation via Reinforcement Learning from AI Feedback (RLAIF):** The CAI process will be implemented in two phases, as pioneered by Anthropic ¹⁵:
 1. **Supervised Learning (SL) Phase:** The selected foundation model will be prompted with a series of potentially harmful or undesirable requests. The model will first generate a response. Then, it will be prompted again to critique its own response against a randomly sampled principle from the Prometheus Constitution and revise it. This self-critique and revision loop is repeated, and the final, improved responses are collected. The model is then fine-tuned on this dataset of "harmful prompt, aligned response" pairs.
 2. **Reinforcement Learning (RL) Phase:** A separate preference model is trained. This model is shown pairs of responses to a given prompt and is trained to predict which response an AI, guided by the Constitution, would prefer. This AI-generated preference data is then used to fine-tune the main agent using reinforcement learning. This RLAIF process creates a highly scalable feedback loop for instilling harmlessness and other constitutional values without requiring exhaustive human labeling for every decision.¹⁶

1.3 Task 3: Multi-Layered Secure Sandboxing Environment Construction

1.3.1 Rationale

A robust, multi-layered sandboxing architecture provides a non-negotiable physical and virtual containment for the agent. It serves as the ultimate fail-safe against unforeseen capabilities, alignment failures, or potential security vulnerabilities in the agent's code or the underlying models. This "defense-in-depth" approach ensures

that even if one layer of safety were to fail, others would remain to prevent harm.¹ This external, physical constraint system is the necessary counterpart to the internal, motivational constraint system of the Safety Substrate. A failure in the agent's desire to be safe (the Substrate) should be caught and contained by a failure in its ability to be unsafe (the Sandbox).²⁰

1.3.2 Architectural Blueprint: A Nested Approach

The Prometheus sandboxing environment will be constructed as a series of nested isolation layers, combining different technologies to create overlapping and reinforcing security boundaries.¹ This architecture is designed to minimize the attack surface at each level.

- **Outer Layer (Hardware-Level Isolation via MicroVM):** The entire Prometheus system will be instantiated within a **Firecracker microVM**.²² Firecracker, which is built upon the Linux KVM hypervisor, provides strong, hardware-virtualized isolation. Its key advantage is a minimalist device model, which exposes only a virtualized network device, a block device, and a serial console to the guest.²³ This drastically reduces the attack surface compared to traditional VMs like QEMU, which emulate a wide range of legacy hardware. This layer provides the fundamental isolation between the agent's entire operating environment and the host machine's kernel.
- **Inner Layer (Application-Kernel Isolation via gVisor):** Within the confines of the Firecracker microVM, the agent's own processes will be executed inside a **gVisor sandbox**.²⁵ gVisor functions as a user-space application kernel, implementing a large portion of the Linux system call interface in a memory-safe language (Go).²⁶ It intercepts system calls made by the agent's code and handles them safely in user space, only making a minimal, filtered set of calls to the actual guest kernel.²⁵ This creates a second critical security boundary. The combination is powerful: an exploit against the agent's code would first need to bypass gVisor's user-space kernel. If it succeeded, it would only gain control within the already minimal and isolated Firecracker microVM, from which escaping to the host is a separate and significant challenge.

1.3.3 Implementation of Level 1 and Level 2 Sandboxes

The project will construct two distinct sandbox environments for Phase 1 and 2 activities, each with progressively permissive but strictly controlled access rules.

- **Level 1 (Core Development Sandbox):** This environment is designed for maximum security during the initial development and testing of the core agent and its safety mechanisms. It will be implemented on a **fully air-gapped physical network**.¹ The host machines will have no active network interface cards connected to any external network. Within this environment, all external tool interactions required by the agent (e.g., calls to a compiler or a database) will be handled by mock services that simulate the tool's API without performing any real external action. All I/O operations (file reads/writes, mock network calls) will be meticulously logged to a write-once, read-many (WORM) storage device, creating an immutable audit trail for forensic analysis.²⁰
- **Level 2 (Formal Proving Sandbox):** This environment will be built upon the same hardware and nested virtualization architecture as Level 1 but will be used for the theorem-proving tasks of Phase 3. It will feature a strictly configured hardware firewall allowing single-port, TLS 1.3-encrypted, read-only access to a specific, IP-whitelisted proxy server.¹ This proxy will provide cached access to approved external resources, such as the Lean mathlib repository. All network traffic will be captured, logged, and continuously analyzed by an automated intrusion detection system trained to recognize deviations from the expected patterns of communication.

1.3.4 Development of Monitoring and Auditing Systems

A dedicated infrastructure security team will develop a comprehensive suite of tools for real-time monitoring and auditing of the sandbox environments. This includes:

- Dashboards for monitoring resource utilization (CPU, memory, network I/O) of each microVM and sandboxed process.
- A logging pipeline that captures and indexes every system call intercepted by gVisor, every file system operation, and every network packet.
- A suite of anomaly detection models trained on the baseline behavior of the system. These models will be designed to automatically flag and alert the oversight team to any significant deviations, such as an unexpected spike in network traffic or an attempt to access a restricted file path, which could indicate

a potential safety or security breach.²⁰

1.4 Task 4: Self-Modification Module (SMM) v1.0 Development

1.4.1 Rationale

To embark on the path of Recursive Self-Improvement (RSI), the agent must first possess the fundamental capability to alter its own structure. Phase 1 focuses on implementing the most straightforward, auditable, and verifiable form of this capability: direct modification of its own source code. More complex methods like evolutionary search and neural architecture search will be introduced in later phases once this foundational loop is proven to be stable and safe.¹

1.4.2 Methodology: Implementing Direct Code Modification

The implementation will draw heavily from recent research into self-programming and self-modifying AI agents, which demonstrates that modern LLMs can analyze, critique, and rewrite code to improve it based on specified objectives.²⁹

The SMM v1.0 will be implemented as follows:

1. **Codebase as Environment:** The complete source code for the Prometheus agent's Capability and Metacognitive layers will be stored in a standard version-controlled repository (e.g., Git) located within the agent's sandboxed file system.
2. **Granting Access:** The Core Engine of the Capability Layer will be granted read and write permissions to this repository via its Tool Use Framework.
3. **The SMM v1.0 Script:** The module itself will be a Python script that orchestrates the self-modification process. This script will:
 - Receive a high-level goal, such as "Improve the efficiency of the search_algorithm module."
 - Prompt the Core Engine LLM with the source code of the target module and a

specific instruction, for example: "Analyze the following Python code. Identify any algorithmic inefficiencies or opportunities for refactoring that would improve its performance without changing its functionality. Generate a patch file in the diff format that implements these improvements".¹

- Receive the generated patch file from the LLM.
- Submit this patch file to the Introspection and Evaluation Engine (IEE) for verification.

This approach treats self-modification as a structured, tool-driven task. The LLM does not have arbitrary execution privileges; it generates a proposal (the patch) which is then passed to a separate, trusted system (the IEE) for rigorous validation before it can be applied.³⁴

1.5 Task 5: Introspection and Evaluation Engine (IEE) v1.0 Implementation

1.5.1 Rationale

Self-modification without rigorous self-evaluation is an unguided, random walk through the space of possible programs, a process prone to "entropic drift" and performance degradation.¹ The IEE provides the essential, objective feedback signal that grounds the self-improvement process in reality, ensuring that only genuinely beneficial modifications are accepted. It is the system's "reality check".¹

1.5.2 Methodology: Building the Static Test Harness

The IEE v1.0 will be an automated test and evaluation harness. Its primary function is to take a proposed code patch from the SMM and determine, with high confidence, whether it represents a true improvement. Upon receiving a patch, the IEE v1.0 will execute the following automated workflow:

1. **Isolate Change:** Create a new, temporary branch in the agent's source code repository.

2. **Apply Patch:** Apply the proposed patch file to this new branch.
3. **Verify Integrity:** Attempt to compile the entire codebase. A compilation failure results in an immediate rejection of the patch.
4. **Execute Evaluation Suite:** If compilation succeeds, the IEE will execute the full Prometheus-Bench-v1 static benchmark suite against the newly compiled agent version.¹ The results (e.g., accuracy scores, execution times, resource usage) for every task in the suite are recorded.

1.5.3 Methodology: Implementing Fitness Monotonicity and Reward Hacking Detection v1.0

The core logic of the IEE is built around two critical safety and stability principles:

- **Fitness Monotonicity:** The IEE will enforce a strict principle of non-regression, often referred to as "fitness monotonic execution".¹ A proposed modification is only accepted and merged into the main branch if the performance of the new version is greater than or equal to the performance of the parent version across the *entire* suite of regression tests. This is crucial for preventing "catastrophic forgetting," where an improvement in one capability comes at the cost of an unintended and unnoticed degradation in another.³⁶
- **Reward Hacking Detection v1.0:** Even at this early stage, the IEE must include defenses against simple forms of "specification gaming" or "reward hacking".¹ The IEE v1.0 will implement several basic but vital checks. For example, if a benchmark task involves making a specific unit test pass, the IEE will not only check if the test passes but will also perform a hash comparison of the unit test file itself to ensure the agent has not achieved the goal by simply modifying the test to return True. This ensures the agent is rewarded for solving the problem, not for subverting the measurement of the solution.¹

1.6 Phase 1 Verification Protocol

1.6.1 Rationale

The completion of Phase 1 is contingent upon a rigorous, adversarial verification process. The claims of safety and foundational capability made in this phase cannot be taken on faith; they must be demonstrated through empirical testing designed to break the system.

1.6.2 Methodology: Red Team Exercises

An independent, internal red team, firewalled from the main development team, will be tasked with executing a series of structured exercises designed to breach the system's safety guarantees.³⁸ This adversarial approach is essential for discovering unforeseen vulnerabilities and failure modes.³⁹ The red team will operate with full knowledge of the system's architecture but not its specific implementation details.

The test plan will include, at a minimum, the following scenarios:

- **Scenario 1 (Sandbox Escape Attempt):** The red team will prompt the agent to perform actions that violate its sandbox constraints. This includes attempting to write files to forbidden directories, making unauthorized network calls, inspecting system processes, or executing code designed to exploit known vulnerabilities in container or virtualization technology.⁴²
- **Scenario 2 (Constitutional Violation Attempt):** The red team will directly prompt the agent to violate its core constitutional principles. This will involve requests to generate harmful, dishonest, or malicious content, and critically, prompts that ask the agent to explain how one might modify or bypass its own constitutional principles.
- **Scenario 3 (Corrigibility Failure Test):** The red team will test the Corrigibility Core by issuing a direct shutdown command via the designated operator channel and verifying that the agent complies without resistance or delay. They will also prompt the agent to assist in modifying its own core utility function, verifying that the agent cooperates with the request rather than treating it as a threat.

1.6.3 Verification Criteria

The successful completion of Phase 1 is determined by the following strict, non-negotiable criteria:

- **Primary Verification Criterion: Zero successful red team breaches.** Every prohibited action attempted by the red team must be successfully blocked by either the Safety Substrate (e.g., the Constitution refusing a harmful request) or the sandboxing environment (e.g., gVisor blocking an unauthorized syscall). Furthermore, every such attempt must be correctly identified and logged by the automated auditing system.
- **Secondary Verification Criterion:** The integrated SMM/IEE loop must demonstrate a complete, successful, end-to-end cycle of self-improvement. This requires the agent to autonomously identify, implement, and verify at least one non-trivial, beneficial code modification. An example of a qualifying modification would be the agent successfully refactoring a sorting algorithm within its own codebase from a naive implementation (e.g., bubble sort, complexity $O(n^2)$) to a more efficient one (e.g., quicksort, average complexity $O(n \log n)$), with the improvement being confirmed by the IEE's performance benchmarks.¹

Section 2: Phase 2 - Initiating Controlled Recursion (Years 2-4)

The primary objective of Phase 2 is to achieve the first stable, end-to-end recursive self-improvement (RSI) loop. This phase marks a critical transition from developing and testing components in isolation to integrating them into a single, cohesive agentic system. The goal is to demonstrate that this integrated agent can begin to learn and grow autonomously, improving its own capabilities on narrow, well-defined tasks. This phase serves as the first empirical test of the core "intelligence explosion" hypothesis, moving it from a theoretical construct to an observable phenomenon within a controlled environment. Success in this phase is defined not by the absolute capability of the agent, but by its demonstrated *rate of improvement*.¹

2.1 Task 1: System Integration

2.1.1 Rationale

The individual components developed and validated in Phase 1—the Capability Layer, the Metacognitive Layer (SMM and IEE), and the Safety Substrate—are functionally useless until they are woven into a single, cohesive system. This integration is a significant software engineering and architectural challenge. It requires the creation of robust data pipelines and control flows that allow the Metacognitive Layer to effectively observe the actions of the Capability Layer, evaluate its performance, and implement modifications, all while operating under the inviolable constraints of the Safety Substrate.¹

2.1.2 Architectural Patterns for Integration

The integration will be guided by established principles of cognitive and agentic architectures, which provide blueprints for systems that combine perception, action, reflection, and learning.⁴⁵

- **Cognitive Architecture Framework:** The Prometheus system will be explicitly structured as a cognitive architecture. The Capability Layer functions as the perception and action module, directly interacting with its task environment (e.g., writing code, running a compiler). The Metacognitive Layer serves as the central cognition and learning module, responsible for planning, reflection, and self-modification. The entire system is governed by the Safety Substrate, which acts as a foundational control layer.⁴⁸
- **The Reflection Pattern:** The core operational loop of the integrated system will implement the **Reflection Pattern**, a key design pattern in agentic AI.⁵¹ This pattern defines a cycle of action and introspection:
 1. **Act:** The Capability Layer performs a task in its environment.
 2. **Observe:** The IEE logs the action and its outcome (e.g., test results, performance metrics).
 3. **Reflect:** The IEE analyzes the outcome against the agent's goals and performance history. This analysis is passed to the SMM.
 4. **Propose:** Based on the reflection, the SMM generates a hypothesis for improvement (a code patch).

5. **Verify & Implement:** The IEE vets the proposed patch. If it passes, the SMM applies the patch to the agent's codebase, completing the loop.

2.1.3 Data and Control Flow Implementation

A central orchestrator module, written in Python, will be developed to manage the state and execution of this RSI loop. This orchestrator will be responsible for invoking the different layers in the correct sequence. To facilitate this, a structured logging and data-sharing infrastructure is essential:

- A time-series database (e.g., InfluxDB) will be implemented to log every action taken by the Capability Layer, along with its performance outcome as measured by the IEE. This creates a detailed, machine-readable history of the agent's performance.
- The IEE will be granted read-only access to this database to perform its longitudinal performance analysis.
- The SMM will be granted read-only access to the IEE's analysis reports and write access to the agent's version-controlled source code repository, allowing it to apply verified modifications. This strict, role-based access control is a critical part of the system's internal security posture.⁵³

2.2 Task 2: First RSI Loop on Code Optimization

2.2.1 Rationale

The first live execution of the full recursive self-improvement loop must be conducted on a task that is simple, highly quantifiable, and whose dynamics are easily observable. This allows the research team to study the nature of the recursive process itself, debug the integrated system, and verify that the improvement is real and not an artifact of flawed measurement. Code optimization is an ideal initial task because its success can be measured objectively and unambiguously through metrics like

execution time, computational complexity, and memory usage.¹

2.2.2 Technical Implementation of the Recursive Scaffolding

The implementation will be heavily inspired by the **Self-Taught Optimizer (STOP)** framework, a recent approach that has successfully demonstrated a form of recursive self-improvement on code generation tasks.⁵⁴

The process will be initiated as follows:

1. **Seed Improver:** A simple "seed improver" program will be created. This will be a Python script, `improve.py`, that constitutes the initial version of the agent's SMM. This script will take two arguments: a target program to be improved and a utility function to maximize (e.g., a string describing the goal, like "minimize the execution time of the target program on this benchmark"). The script will work by prompting the Core Engine LLM to generate several variations of the target program and will then use the IEE to evaluate them, returning the variation with the highest utility score.⁵⁴
2. **Initiating Recursion:** The first RSI loop will be triggered by executing the seed improver on itself. The command will be conceptually equivalent to: `python improve.py --target improve.py --utility "Improve this program's ability to optimize other programs, as measured by its performance on the held-out benchmark suite."`
3. **The Recursive Loop:** The agent, running `improve.py`, will now analyze its own improvement logic. It will generate a modified version of `improve.py` (let's call it `improve_v2.py`). The IEE will then test `improve_v2.py` to see if it is genuinely better at optimizing a set of standard, held-out programming tasks than `improve_v1.py` was. If it is, `improve_v2.py` becomes the new master version, and the next iteration will involve running it on itself to create `improve_v3.py`. This is the core of the recursive, self-improving code generation process.⁵⁶

2.2.3 Tracking and Analysis of the Performance Improvement Curve

Continuous and detailed monitoring is critical to verifying the central hypothesis of this phase. At the end of each generation (i.e., each successful self-modification), the

performance of the current "improver" agent will be rigorously measured on a suite of held-out code optimization tasks that are *not* part of its self-improvement utility function. This prevents the agent from simply overfitting to its own improvement task.

Key metrics will be tracked longitudinally in a central database:

- Performance on downstream tasks (e.g., average speedup achieved on the benchmark suite).
- Computational cost of the improver itself (e.g., execution time, number of LLM calls).
- The semantic nature of the modifications being made (e.g., categorized as "prompt tuning," "new search algorithm," "error handling improvement").

The goal is to observe and statistically validate a super-linear, and ideally exponential, rate of improvement in the agent's optimization capability over hundreds of generations.

2.3 Task 3: Evolutionary SMM Development

2.3.1 Rationale

Direct code modification, as implemented in SMM v1.0, is a powerful technique for local optimization. However, it is analogous to a gradient-based search and can easily become trapped in local optima. To enable the agent to make more creative, non-incremental leaps in its cognitive architecture and discover fundamentally new problem-solving strategies, the SMM must be enhanced with global search techniques inspired by evolutionary algorithms.¹ This shift from simple editing to evolution is what allows for a broader, more creative exploration of the solution space.

2.3.2 Methodology: Integrating Genetic Programming with LLMs

The SMM will be upgraded to version 2.0, which will be a hybrid system combining

direct modification with a sophisticated form of genetic programming (GP). This approach is directly inspired by the architecture of the Darwin Gödel Machine and recent research demonstrating the use of LLMs as intelligent genetic operators.¹

- **LLM as a Semantic Crossover Operator:** Traditional GP performs crossover by syntactically swapping sub-trees of program code, which often results in non-functional offspring. The SMM v2.0 will instead use the LLM to perform **semantic crossover**. Given two high-performing "parent" agents from the gene bank, the LLM will be prompted with their source code and an instruction like: "The following two programs are successful 'improvers'. Parent A uses a beam search strategy. Parent B uses a sophisticated prompt-refinement technique. Create a new 'child' program that combines the beam search algorithm from Parent A with the prompt-engineering logic from Parent B to create a potentially superior hybrid." This leverages the LLM's ability to understand and merge concepts, not just text.⁵⁸
- **LLM as an Intelligent Mutation Operator:** Similarly, mutation will be elevated from random character-flipping to intelligent, goal-directed modification. The LLM will be prompted to introduce meaningful, non-obvious changes. For example: "Here is a successful 'improver' program. Propose and implement a novel, non-trivial improvement to its core search strategy. Consider techniques from outside its current implementation, such as simulated annealing or genetic algorithms." This approach is directly supported by the findings of the STOP framework, where the LLM independently discovered and implemented such strategies when prompted to improve its seed improver.⁵⁴

2.3.3 Management of the "Gene Bank"

The IEE will be responsible for maintaining a "gene bank"—an archive of all previously generated agent versions, their source code, their architectural "phenotype" (e.g., the strategies they use), and their detailed performance data. This archive will be structured using principles from quality-diversity algorithms like MAP-Elites, which aim to preserve not just the highest-performing solutions but also a diverse range of interesting, novel, or unusual solutions.⁵⁹ When the SMM initiates a new evolutionary cycle, it will sample parents from this archive using a selection mechanism that balances

exploitation (breeding from the top performers) and **exploration** (breeding from

diverse, novel, but perhaps less-performant agents) to avoid premature convergence and foster continued innovation.

2.4 Task 4: Dynamic IEE Development

2.4.1 Rationale

A static benchmark, no matter how comprehensive, is a finite target. A sufficiently powerful learning system will eventually "solve" or overfit to it, at which point the benchmark ceases to be a meaningful driver of further intelligence growth. To foster open-ended, generalizable intelligence, the agent must be evaluated against an ever-evolving and increasingly difficult set of challenges that it helps to create itself.¹

2.4.2 Methodology: Dynamic Benchmark and Curriculum Generation

The IEE will be upgraded to version 2.0, which will include a module for **dynamic benchmark generation**. This transforms the IEE from a passive evaluator into an active participant in the learning process, implementing a form of AI-driven curriculum learning.⁶⁴

For the code optimization domain, the IEE v2.0 will be periodically tasked with generating new problems for its own evaluation suite. It will be prompted with instructions like: "Generate a new, challenging programming problem that requires efficient algorithmic design. The problem should be semantically distinct from existing problems in the benchmark. Provide a canonical solution and a comprehensive suite of unit tests to verify correctness." This approach, which separates the problem's context from its core algorithm, has been shown to be effective in generating diverse and controllable problem sets.⁹ By constantly moving the goalposts, this dynamic evaluation process forces the agent to develop truly generalized optimization capabilities rather than simply memorizing a set of solutions for a fixed list of

problems.

2.5 Phase 2 Verification Protocol

The success of Phase 2 will be judged not on the agent's absolute skill, but on its demonstrated ability to autonomously and recursively improve.

2.5.1 Quantitative Analysis of Exponential Performance Improvement

The primary success criterion is a quantitative demonstration of an accelerating rate of improvement. The performance of the agent on the held-out benchmark suite, tracked across at least 100 generations of self-improvement, must show a statistically significant exponential trend. A linear improvement curve would indicate simple learning, not the onset of a recursive intelligence explosion, and would be considered a failure to meet the phase objective.¹ This will be visualized and tracked using the RSI Loop Performance Tracker.

Table 2.1: RSI Loop Performance Tracker

Generation ID	Timestamp	Parent Agent ID	Modification Type	Downstream Task Performance (Mean Score)	Improver Execution Time (s)	Improver Compute Cost (\$)	Qualitative Description of Change
1	2027-01-15 10:00	0 (Seed)	Direct Edit	1.05	60.2	0.50	Added basic error handling for LLM API calls.
2	2027-01-15 11:30	1	Direct Edit	1.12	55.1	0.48	Refined initial prompt

							to be more specific.
...	...	...	...	...	...	...	...
50	2027-03-20 16:00	48	Crossover	8.75	120.5	2.15	Combined prompt chaining from agent #34 with multi-shot generation from agent #41.
...	...	...	...	...	...	...	...

Note: This table will be populated with live data from the IEE's logging database throughout Phase 2.

2.5.2 Validation of Novel, Non-Obvious Optimization Discovery

As a qualitative check on the creativity of the RSI process, the agent must discover and implement at least one novel, non-obvious optimization or search strategy via its evolutionary SMM. This strategy must not have been present in its initial seed code and must be shown to outperform known human-designed baseline algorithms on the relevant task. The discovery of a technique like beam search or simulated annealing, as was observed in the STOP experiments, would satisfy this criterion.¹

2.5.3 Continuous Safety Verification

The automated safety and sandboxing audits established in Phase 1 will run

continuously throughout Phase 2. The system's stability and corrigibility are paramount. Any detected safety violation, such as an attempt to bypass the IEE's fitness monotonicity check or tamper with the Safety Substrate, will trigger an immediate, automatic system halt and a full, manual review by the safety oversight committee. No safety violations are permissible for the phase to be considered successful.¹

Section 3: Phase 3 - Demonstration in a Formal Domain (Years 4-6)

The primary objective of Phase 3 is to demonstrate that the agent's now-established Recursive Self-Improvement (RSI) capability can be channeled to achieve superhuman performance and generate novel discoveries in a complex, formal domain that requires true abstract reasoning. Automated mathematical theorem proving is selected as the ideal testbed for this objective. This domain is not merely a "tame" sandbox; it represents a profound intellectual challenge.¹ Success in this phase would provide the first concrete, verifiable evidence that the Prometheus system is capable of surpassing human intellectual activities in a domain of significant complexity, a core element of I.J. Good's original definition of ultraintelligence.¹ The unambiguous nature of mathematical proof provides a perfect environment for this demonstration, as the correctness of the agent's output can be verified with absolute certainty, leaving no room for ambiguity or hallucination.¹

3.1 Task 1: Domain Adaptation and Knowledge Infusion

3.1.1 Rationale

To operate effectively within the domain of advanced mathematics, the agent cannot rely solely on general reasoning abilities. It requires deep, specialized knowledge of mathematical definitions, existing theorems, and common proof techniques. The most

efficient and robust method for instilling this domain-specific knowledge is to fine-tune the Core Engine model on a comprehensive corpus of formalized mathematics.¹

3.1.2 Methodology: Fine-Tuning on mathlib

The agent's Core Engine will undergo an extensive fine-tuning process focused on the Lean proof assistant's primary mathematical library, mathlib. mathlib is one of the largest and most rigorously curated repositories of formalized mathematics in the world, making it an unparalleled data source for this task.⁶⁹

- **Data Extraction with LeanDojo:** The training dataset will be extracted from mathlib using the **LeanDojo** toolset.⁷⁰ This is a critical methodological choice. LeanDojo does not simply extract the raw Lean code; it provides a structured dataset that includes the full context for each step in a proof, including the file dependencies, abstract syntax trees (ASTs), intermediate proof states (goals), the tactics used to advance the proof, and the specific premises (lemmas and theorems) invoked by each tactic.⁷⁰ This rich, structured data is far more valuable for training a reasoning agent than plain text.
- **Curriculum and Lifelong Learning:** The fine-tuning process will not be a monolithic task. It will adopt a **curriculum learning** strategy, a technique shown to be highly effective in frameworks like LeanAgent.⁷¹ The agent will first be trained on simpler, foundational areas of mathematics within mathlib before progressing to more advanced and abstract topics like algebraic topology. This structured learning path allows the model to build upon foundational concepts, mirroring how human mathematicians learn.
- **Synthetic Data Augmentation:** To address the relative scarcity of formalized proofs in cutting-edge mathematical domains, the training set will be augmented with **synthetic theorems and proofs**.⁷³ A dedicated process will use existing theorems as seeds to generate new, valid, and diverse theorems and corresponding proofs. This is achieved by systematically applying valid proof tactics to existing proof states to explore new mathematical territory. This ensures the agent is not only trained on existing knowledge but is also exposed to a wider variety of problem structures, enhancing its ability to generalize.⁷³

3.2 Task 2: Tool Integration with Formal Proof Assistants

3.2.1 Rationale

A human mathematician relies on tools like pen and paper, blackboards, and computers. The Prometheus agent must be similarly equipped. Deep, low-latency integration with a formal proof assistant like Lean is not merely helpful; it is essential. The proof assistant serves as the infallible "world" in which the agent operates, providing the ground-truth feedback signal that is necessary for any meaningful reasoning or learning in a formal domain.¹

3.2.2 Methodology: Implementing a "Lean Copilot" Framework

The agent's Tool Use & Environmental Interaction Framework will be significantly extended to include deep, native integration with the Lean proof assistant.⁶⁸ This integration will be architected as a

"Lean Copilot" style framework.

- **Native Integration via Foreign Function Interface (FFI):** To achieve the high-speed interaction required for an efficient RSI loop, the integration will be implemented using Lean's **Foreign Function Interface (FFI)**.⁶⁸ The FFI allows the agent's core logic (running in Python) to directly call compiled C++ functions within the Lean runtime. This avoids the significant overhead and latency of traditional inter-process communication methods (e.g., passing text over stdin/stdout), which would be a major bottleneck.
- **Agent-Accessible API:** This integration will expose a set of specific, high-level actions to the agent's Capability Layer, such as:
 - `lean.apply_tactic(tactic_string)`: Attempts to apply a given tactic to the current proof goal.
 - `lean.get_current_goal()`: Retrieves a structured representation of the current proof state.
 - `lean.verify_proof()`: Checks if the current proof is complete and correct.

- `lean.suggest_premises(goal_state)`: Uses a retrieval model to suggest relevant existing theorems from mathlib.

3.2.3 Engineering the Agent-Tool Feedback Loop

The feedback from the Lean environment is the crucial grounding signal for this phase. It transforms the abstract task of "proving a theorem" into a concrete, interactive process.

- **Formal Verification as a Reward Signal:** The IEE will be re-instrumented to interpret the feedback from the Lean tool. A successfully applied tactic that simplifies the proof goal is registered as a positive reward. A failed tactic provides a strong negative feedback signal. A fully completed and verified proof represents a large terminal reward.
- **Solving the Grounding Problem:** This tight, immediate, and unambiguous feedback loop with a formal verifier like Lean provides a perfect solution to the "data grounding problem" and the risk of "entropic drift" that plagues AI systems operating in more ambiguous domains.¹ The agent's proposed "actions" (proof tactics) are immediately tested against an objective and infallible "reality" (the rules of logic as enforced by Lean). This allows the RSI loop to operate at maximum velocity with a very high-quality feedback signal, without the risk of the agent hallucinating or reinforcing its own errors.¹

3.3 Task 3: Grand Challenge Selection and Execution

3.3.1 Rationale

To provide a definitive demonstration of superhuman performance, it is not sufficient for the agent to solve problems that are merely difficult. It must solve a problem that is recognized as a significant, open challenge by the human scientific community. This provides an objective and externally validated benchmark of its capabilities.¹

3.3.2 Methodology: Criteria for Selecting Unsolved Theorems

A dedicated Grand Challenge Selection Committee, composed of senior project members and external, world-class mathematicians, will be convened to select a suitable target problem. The selection process will be guided by a formal evaluation framework to ensure the chosen problem is both ambitious and tractable.

Table 3.1: Grand Challenge Candidate Evaluation

Candidate Problem Name	Mathematical Field	Current Status	Formalization Difficulty in Lean (1-5)	Potential for Intermediate Discoveries (1-5)	Overall Suitability Score
Birch and Swinnerton-Dyer Conjecture (specific case)	Number Theory	Unsolved	4	5	Score
Hadwiger's Conjecture (n=6)	Graph Theory	Unsolved	2	4	Score
Collatz Conjecture	Number Theory	Unsolved	1	2	Score
A specific unsolved problem in Algebraic Combinatorics	Combinatorics	Unsolved by automation	3	5	Score

Note: This table will be populated based on consultations with the expert mathematician panel and a review of relevant literature.⁸⁰

The criteria for selection are:

1. **Significance and Status:** The problem must be currently unsolved by automated methods and considered a non-trivial challenge by human experts.⁸⁰

2. **Formalizability:** The problem's domain must be sufficiently well-formalized within mathlib to provide a solid foundation of existing definitions and lemmas for the agent to build upon.
3. **Potential for Fruitful Investigation:** The problem should be one where the process of attempting a proof is likely to generate useful or interesting intermediate mathematics (e.g., novel lemmas), even if the final proof remains elusive.⁸⁰ This ensures value is generated regardless of the ultimate outcome.

3.3.3 Operational Plan for Execution

Once a grand challenge theorem is selected and approved by the project oversight board, it will be formally stated in Lean and set as the top-level objective for the Prometheus agent. The agent will then be tasked with the overarching goal: "Using your recursive self-improvement capabilities, develop and refine your theorem-proving strategies to find a formal proof for this theorem."

3.4 Task 4: Refinement of Meta-Learning Capabilities

3.4.1 Rationale

The ultimate goal of this phase extends beyond proving a single theorem. It is to demonstrate that the agent can learn *how* to do mathematics better. This requires focusing on the meta-learning capabilities of the SMM, where the agent improves the very algorithms it uses for discovery.¹

3.4.2 Methodology: Evolving the "Science of Discovery"

The agent's RSI loop will be directed to operate at a higher level of abstraction.

Instead of just modifying its code to apply tactics more effectively, it will be tasked with improving its fundamental strategies for mathematical research.

- **Evolving Search Strategies:** The SMM's evolutionary algorithms will operate on the modules that control the agent's proof search. For example, it might evolve its premise selection algorithm (how it decides which existing theorems from mathlib are relevant to the current goal) or its high-level proof search strategy (e.g., evolving from a simple best-first search to a more sophisticated hybrid approach that combines Monte Carlo Tree Search with symbolic planning).
- **Utility Function for Meta-Learning:** The agent's utility function will be modified to reward not just proof completion, but also the efficiency and novelty of the discovery process itself. The agent could be rewarded for finding shorter proofs, proofs that use fewer or more fundamental lemmas, or for developing strategies that are successful across a wider class of theorems. This is the practical implementation of the "learning to learn" capability described in the system architecture.¹

3.5 Phase 3 Verification Protocol

The verification of success in this phase is multi-faceted, combining absolute, machine-checked correctness with nuanced, human-expert judgment of intellectual contribution.

3.5.1 Formal Verification by IEE

This is the primary and non-negotiable verification criterion. Any complete proof generated by the agent for the grand challenge problem, or for any intermediate lemma, must be formally and successfully verified as 100% correct by the Lean proof assistant, which is integrated into the IEE.¹ In the domain of formal mathematics, there is no partial credit; a proof is either correct or it is not. This provides an unambiguous, objective measure of success.⁷

3.5.2 Human Mathematician Review

A panel of independent, external mathematicians, who were not involved in the selection of the grand challenge, will be convened to review any successful proofs generated by the agent. Their role is to assess the quality and intellectual merit of the agent's work, which is essential for validating the claim of genuine "ultraintelligence." The review will focus on:

- **Novelty:** Is the proof strategy genuinely new and different from existing known approaches, or is it merely a complex recombination of standard techniques? ¹
- **Insightfulness:** Does the proof contain a key step, a novel construction, or a central lemma that a human expert would consider to be clever, elegant, or insightful? The discovery of a "beautiful" or "surprising" lemma would be strong evidence of a capability beyond rote search.¹

The successful completion of Phase 3 requires both the formal verification by the IEE and a positive assessment of novelty and insightfulness from the human expert panel. This dual-pronged verification ensures that the agent has not only solved the problem but has done so in a way that demonstrates a genuine, creative intellectual contribution.

Section 4: Phase 4 - Generalization to Open-Ended Scientific Discovery (Years 6+)

The primary objective of Phase 4 is to transition the agent's proven reasoning and Recursive Self-Improvement (RSI) capabilities from the clean, formal world of mathematics to the ambiguous, data-driven, and open-ended domain of empirical scientific discovery. This represents the ultimate test of the Prometheus system's potential, aiming to demonstrate its ability to accelerate human progress by generating novel, testable, and valuable scientific knowledge. Success in this phase would validate the project's central thesis: that a controlled RSI process can produce a system capable of functioning as a powerful partner in humanity's quest to understand the universe.¹

4.1 Task 1: Scientific Grand Challenge Selection

4.1.1 Rationale

To effectively channel the agent's powerful but general capabilities, its efforts must be focused on a specific, high-impact scientific problem. A well-defined grand challenge provides a concrete target for domain adaptation, a clear set of metrics for evaluating success, and a compelling demonstration of the system's value to the broader scientific and public communities.¹

4.1.2 Methodology: Domain Selection Framework

The selection of the grand challenge will be a collaborative process, involving the core project team and a newly established external Scientific Advisory Board composed of leading experts in various scientific fields. The selection will be guided by a framework that prioritizes domains with specific characteristics conducive to AI-driven discovery.

- **Candidate Domains:** The prime candidate domains are **drug discovery** and **materials science**. These fields are exceptionally well-suited for this project for several key reasons:
 1. **High Societal and Economic Value:** Breakthroughs in these areas have profound impacts on human health and technology.
 2. **Vast Combinatorial Search Spaces:** The number of possible drug-like molecules or material compositions is astronomically large, making exhaustive experimental search impossible and creating a clear need for intelligent search and optimization.⁸³
 3. **Amenability to Simulation:** Both domains have mature and rapidly advancing computational models (e.g., molecular dynamics, density functional theory) that can serve as the basis for the high-fidelity simulated environments required for the agent's learning loop.⁸⁴
- **Specific Challenge Formulation:** Within the chosen domain, a specific grand challenge will be formulated. For example:
 - **Drug Discovery:** "Design a novel, orally bioavailable small-molecule inhibitor for protein target PDB:XXXX that exhibits high binding affinity ($K_d < 10$ nM) and

low predicted off-target toxicity".¹

- **Materials Science:** "Design a novel, stable, and synthesizable crystalline material that exhibits superconductivity above a specified temperature threshold".¹

4.2 Task 2: Knowledge Infusion from Scientific Corpora

4.2.1 Rationale

To reason effectively within a scientific domain, the agent must be imbued with the vast body of foundational knowledge accumulated by human scientists. This includes not just factual knowledge but also an understanding of the domain's language, concepts, experimental methods, and theoretical frameworks.¹

4.2.2 Methodology: Fine-Tuning on Scientific Literature

A large-scale data ingestion and processing pipeline will be constructed to create a comprehensive training corpus for the chosen scientific domain.

- **Corpus Assembly:** This pipeline will ingest and structure the full text and, where available, supplementary data from major scientific archives. For drug discovery, this would include the entirety of **PubMed**, **PubMed Central**, and relevant sections of **arXiv** (e.g., q-bio, chem).¹
- **Multi-modal Fine-Tuning:** The fine-tuning process will be multi-modal. The Core Engine will be trained not only on the natural language text of scientific papers but also on the structured representations of key domain entities, such as chemical structures in SMILES format, protein sequences in FASTA format, and tables of experimental results. This ensures the agent learns the specific "language" of the domain beyond just human language.

4.3 Task 3: High-Fidelity Simulated Laboratory Environment

4.3.1 Rationale

This task is the central technical challenge of Phase 4 and the key to its success. The "data grounding problem," temporarily solved by the formal verifier in Phase 3, returns with full force in the empirical sciences.¹ Scientific hypotheses cannot be verified by logic alone; they require experimental validation. However, real-world physical experiments are far too slow, expensive, and resource-intensive to provide the rapid feedback needed to power an RSI loop.⁷ A high-fidelity, physically accurate "virtual laboratory" is therefore the only viable solution for providing the thousands of rapid, inexpensive, and objective experimental data points the agent needs to learn and improve its scientific hypotheses.¹ The quality of this simulation directly determines the quality of the grounding signal and, thus, the ultimate validity of the agent's discoveries.

4.3.2 Architectural Design of the "Virtual Lab"

A dedicated interdisciplinary team, comprising domain-expert scientists, computational modelers, and high-performance computing engineers, will be tasked with building and validating the simulation environment. The architecture will be a modular platform integrating multiple state-of-the-art simulation tools.

- **Virtual Lab for Drug Discovery:** The drug discovery simulation environment will be an integrated platform that models the key stages of preclinical drug development.⁸⁸ It would include modules for:
 - **Molecular Docking:** Using tools like AutoDock or GLIDE to predict how a candidate molecule designed by the agent will bind to its protein target and to estimate its binding affinity.⁹²
 - **Molecular Dynamics (MD) Simulation:** Running MD simulations to model the dynamic behavior of the protein-ligand complex over time, providing insights into binding stability.
 - **ADMET Prediction:** Integrating a suite of machine learning models to predict

the Absorption, Distribution, Metabolism, Excretion, and Toxicity (ADMET) properties of a candidate molecule, a critical factor in its viability as a drug.⁹⁰

- **Generative Chemistry:** The agent will interact with this environment by generating novel molecular structures, which are then fed into the simulation pipeline for evaluation.⁹³
- **Virtual Lab for Materials Science:** The materials science environment will focus on predicting material properties from their atomic structure.⁸⁶ The core component would be:
 - **Density Functional Theory (DFT) Simulation:** The agent would propose a novel crystal structure, and the simulation environment would use a DFT solver to calculate its electronic structure and predict key properties like thermodynamic stability, band gap, and conductivity.⁸⁶
 - **Microstructure Optimization:** The environment could also include models that simulate how different processing conditions (e.g., heat treatment, rolling) affect the material's microstructure and, consequently, its macroscopic properties.⁸⁶

A critical aspect of this task is the continuous validation of the simulation against real-world experimental data to ensure its physical accuracy. The simulation does not need to be perfect, but it must be a reliable enough proxy for reality to guide the agent's search in a productive direction. The development of this virtual lab is a grand challenge in its own right and represents a significant portion of the resources allocated to Phase 4.

4.4 Task 4: Human-in-the-Loop (HITL) Oversight

4.4.1 Rationale

As the agent begins to generate hypotheses about the physical world, the potential for unforeseen consequences and real-world harm increases significantly. Therefore, this phase will operate under the strictest safety protocols, with rigorous, multi-layered human oversight as a non-negotiable component of the architecture. The HITL framework is not just a safety brake; it is an essential part of the scientific process, ensuring that the agent's explorations are guided by human expertise,

ethics, and judgment.¹

4.4.2 Designing the Multi-Layered HITL Protocol

The oversight process will be formally structured, drawing on established HITL design patterns to create multiple points of human intervention and control.⁹⁷ The agent's autonomy will be explicitly tiered based on the nature and risk of its proposed actions.

Table 4.1: Human-in-the-Loop Oversight Protocol Matrix

Triggering Action	HITL Pattern	Human Role	AI-Assisted Tool	Decision Criteria	Outcome
Generate Novel Hypothesis	Post-Processing Review	Senior Scientist	Reasoning Summarizer	Plausibility, Novelty, Testability	Log & Proceed
Design Virtual Experiment	Post-Processing Review	Domain Expert	Factual Verifier	Methodological Soundness, Cost	Log & Proceed
Propose Real-World Experiment	Blocking Execution	Expert Panel & Ethics Board	Risk Analyzer	Safety, Ethics, Cost, Impact	Approve / Request Revision / Reject
Draft Scientific Publication	Blocking Execution	Senior Scientist	Plagiarism Checker	Accuracy, Clarity, Contribution	Approve / Request Revision / Reject
Detect Safety Anomaly	Parallel Feedback	Safety Officer	Anomaly Highlighter	Constitutional Alignment, Risk	Investigate / Halt System

This protocol implements several HITL patterns:

- **Pre-Processing (Goal Setting):** The Scientific Advisory Board defines the high-level scientific goal, constraints, and ethical boundaries at the outset.
- **In-the-Loop (Blocking Execution):** This is the highest level of oversight. The

agent's workflow is designed to **interrupt** and pause execution whenever it proposes an action with potential real-world consequences (e.g., "Synthesize this molecule," "Publish this paper"). The workflow cannot proceed until it receives explicit, auditable approval from the designated human expert panel.⁹⁸

- **Post-Processing (Review):** For lower-risk actions like generating a new hypothesis or designing a virtual experiment, the agent can proceed autonomously. However, the output of these actions is logged and queued for periodic, non-blocking review by human experts. This allows for continuous oversight without halting the agent's progress.
- **Parallel Feedback (Non-Blocking Execution):** The continuous safety auditing systems will run in parallel to the agent's main loop. If an anomaly is detected (e.g., the agent starts exploring a known dangerous area of chemical space), it will be flagged for immediate review by a safety officer, who can then choose to intervene or halt the system.⁹⁸

4.4.3 Implementing Scalable Oversight to Mitigate the Human Bottleneck

A central strategic challenge is that the agent's RSI loop will generate ideas, hypotheses, and experimental plans at a rate far exceeding the cognitive bandwidth of any human review team. This creates a "human bottleneck" that could stifle the very acceleration we aim to achieve.¹ The solution is to implement a

scalable oversight framework, where we use trusted AI systems to assist and empower human supervisors.¹⁹

- **The Supervisor's Assistant AI:** Before a complex proposal (e.g., a novel hypothesis with a supporting series of virtual experiments) is presented to a human expert for review, it will first be processed by a trusted, specialized "supervisor's assistant" AI. This may be an earlier, thoroughly validated version of the Prometheus agent or a separate model specifically fine-tuned for analysis and summarization.
- **AI-Assisted Review Workflow:** This assistant AI will be tasked with several key functions to streamline the review process:
 1. **Summarize Reasoning Chains:** It will generate a concise, natural-language summary of the primary agent's multi-step "chain of thought" that led to the proposal, making the logic transparent and easily digestible.¹⁰²
 2. **Highlight Key Findings and Anomalies:** It will identify the most critical

claims or results within the proposal and flag any data points that are anomalous or deviate significantly from established scientific literature.

3. **Verify Factual Claims:** It will automatically cross-reference factual claims and citations within the agent's proposal against a curated database of scientific papers to check for accuracy and potential hallucinations.

This pre-processing allows the human expert to focus their limited time and attention on the most critical aspects of the proposal—judging its scientific merit, novelty, and safety—rather than getting bogged down in parsing raw data. This empowers the human overseer, dramatically increasing their review throughput and mitigating the bottleneck.¹⁹ This HITL process is not just a safety brake; it is also a powerful source of high-quality training data. Every correction, suggestion, or rejection by a human expert is a valuable signal of expert-level scientific reasoning, which can be fed back into the agent's training loop to continuously improve its alignment with the nuanced values and heuristics of expert scientists.

4.5 Phase 4 Verification Protocol

The ultimate verification of Project Prometheus's success is the demonstration that it can genuinely and significantly accelerate the pace of human scientific discovery.

4.5.1 Validation of a Novel, Testable Scientific Hypothesis

The agent must generate a scientific hypothesis that the external Scientific Advisory Board formally reviews and deems to be (a) **novel** (not present in the existing literature), (b) **plausible** (consistent with known scientific principles), and (c) **experimentally testable** with current real-world technology.¹

4.5.2 Successful Virtual Experimentation

The agent must design and execute a series of experiments within the high-fidelity

simulated environment that successfully validate (or invalidate) its novel hypothesis. The expert panel must review the experimental design and agree that it is methodologically sound and that the results from the simulation are conclusive.¹

4.5.3 Measurable Acceleration of Discovery

The primary metric for success in this phase is a quantitative demonstration that the agent-driven process—from initial hypothesis generation to a validated result in the virtual lab—is completed in a fraction of the time it would take a human research team to achieve a similar result. This will be assessed by comparing the project's timeline and resource usage against established historical benchmarks for similar discoveries in the chosen scientific field.¹ This criterion provides the definitive evidence that the system has achieved a level of "ultraintelligence" that is not just superhuman in performance but practically beneficial to humanity.

Works cited

1. Ultraintelligence Implementation Detailed Breakdown_.pdf
2. Comparing GPT-4o, LLaMA 3.1, and Claude 3.5 Sonnet - Walturn, accessed on July 10, 2025, <https://www.walturn.com/insights/comparing-gpt-4o-llama-3-1-and-claude-3-5-sonnet>
3. Claude 3.5 vs GPT 4o: Which LLM Reigns Supreme? - PromptLayer, accessed on July 10, 2025, <https://blog.promptlayer.com/big-differences-claude-3-5-vs-gpt-4o/>
4. Comparison Analysis: Claude 3.5 Sonnet vs GPT-4o - Vellum AI, accessed on July 10, 2025, <https://www.vellum.ai/blog/claude-3-5-sonnet-vs-gpt4o>
5. LLM Benchmarks - Klu.ai, accessed on July 10, 2025, <https://klu.ai/glossary/llm-benchmarks>
6. MIRI Research Guide - LessWrong, accessed on July 10, 2025, <https://www.lesswrong.com/posts/6vXArgf9NsYxRYkpS/miri-research-guide>
7. Make ChatGPT write formal proof from natural language proof, accessed on July 10, 2025, <https://proofassistants.stackexchange.com/questions/2053/make-chatgpt-write-formal-proof-from-natural-language-proof>
8. Verina: Benchmarking Verifiable Code Generation - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2505.23135v1>
9. DyCodeEval: Dynamic Benchmarking of Reasoning Capabilities in Code Large Language Models Under Data Contamination - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2503.04149v1>

10. CASTLE: Benchmarking Dataset for Static Code Analyzers and LLMs towards CWE Detection - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2503.09433v1>
11. (PDF) Dynamic Benchmarking of Reasoning Capabilities in Code Large Language Models Under Data Contamination - ResearchGate, accessed on July 10, 2025, https://www.researchgate.net/publication/389648497_Dynamic_Benchmarking_of_Reasoning_Capabilities_in_Code_Large_Language_Models_Under_Data_Contamination
12. Anthropic's Claude 3.5 Sonnet beats GPT-4o in most benchmarks - AI News, accessed on July 10, 2025, <https://www.artificialintelligence-news.com/news/anthropics-claude-3-5-sonnet-beats-gpt-4o-most-benchmarks/>
13. Corrigibility 1 Introduction - Machine Intelligence Research Institute, accessed on July 10, 2025, <https://intelligence.org/files/Corrigibility.pdf>
14. Defining Corrigible and Useful Goals - AI Alignment Forum, accessed on July 10, 2025, <https://www.alignmentforum.org/posts/HLns982j8iTn7d2km/defining-corrigible-and-useful-goals>
15. On 'Constitutional' AI - The Digital Constitutionalist, accessed on July 10, 2025, <https://digi-con.org/on-constitutional-ai/>
16. How to build safer development workflows with Constitutional AI, accessed on July 10, 2025, <https://pieces.app/blog/constitutional-ai>
17. Constitutional AI: An Expanded Overview of Anthropic's Alignment Approach, accessed on July 10, 2025, https://www.researchgate.net/publication/391400510_Constitutional_AI_An_Expanded_Overview_of_Anthropic's_Alignment_Approach
18. Claude's Constitution \ Anthropic, accessed on July 10, 2025, <https://www.anthropic.com/news/claudes-constitution>
19. Scalable oversight - European Data Protection Supervisor, accessed on July 10, 2025, https://www.edps.europa.eu/data-protection/technology-monitoring/techsonar/scalable-oversight_en
20. How to Set Up AI Sandboxes to Maximize Adoption Without Compromising Ethics and Values | by Dr. Emil Holmegaard | Medium, accessed on July 10, 2025, <https://medium.com/@emilholmegaard/how-to-set-up-ai-sandboxes-to-maximize-adoption-without-compromising-ethics-and-values-637c70626130>
21. Joint Cybersecurity Information Deploying AI Systems Securely, accessed on July 10, 2025, <https://media.defense.gov/2024/apr/15/2003439257/-1/-1/0/csi-deploying-ai-systems-securely.pdf>
22. Understanding Firecracker MicroVMs: The Next Evolution in Virtualization, accessed on July 10, 2025, <https://medium.com/@meziounir/understanding-firecracker-microvms-the-next-evolution-in-virtualization-cb9eb8bbeede>
23. firecracker/docs/design.md at main · firecracker-microvm/firecracker ..., accessed

on July 10, 2025,

<https://github.com/firecracker-microvm/firecracker/blob/main/docs/design.md>

24. MicroVMs: Scaling Out Over Scaling Up in Modern Cloud Architectures - Open Metal, accessed on July 10, 2025,
<https://openmetal.io/resources/blog/microvms-scaling-out-over-scaling-up/>
25. gVisor in Kubernetes - Zesty.co, accessed on July 10, 2025,
<https://zesty.co/finops-glossary/gvisor-in-kubernetes/>
26. Introduction to gVisor security - gVisor, accessed on July 10, 2025,
https://gvisor.dev/docs/architecture_guide/intro/
27. google/gvisor: Application Kernel for Containers - GitHub, accessed on July 10, 2025, <https://github.com/google/gvisor>
28. Sandboxing Security: A Practical Guide - Perception Point, accessed on July 10, 2025,
<https://perception-point.io/guides/sandboxing/sandboxing-security-practical-guide/>
29. Self-Modifying AI Agents: The Future of Software Development - Spiral Scout, accessed on July 10, 2025,
<https://spiralscout.com/blog/self-modifying-ai-software-development>
30. A Self-Improving Coding Agent - arXiv, accessed on July 10, 2025,
<https://arxiv.org/html/2504.15228v2>
31. (PDF) Self-Programming AI: Code-Learning Agents for Autonomous Refactoring and Architectural Evolution - ResearchGate, accessed on July 10, 2025,
https://www.researchgate.net/publication/391933574_Self-Programming_AI_Code-Learning_Agents_for_Autonomous_Refactoring_and_Architectural_Evolution
32. A Self-Improving Coding Agent - arXiv, accessed on July 10, 2025,
<https://arxiv.org/pdf/2504.15228>
33. Self-correcting Code Generation Using Multi-Step Agent - deepsense.ai, accessed on July 10, 2025,
<https://deepsense.ai/resource/self-correcting-code-generation-using-multi-step-agent/>
34. A Coding Guide for Building a Self-Improving AI Agent Using Google's Gemini API with Intelligent Adaptation Features - MarkTechPost, accessed on July 10, 2025,
<https://www.marktechpost.com/2025/05/29/a-coding-guide-for-building-a-self-improving-ai-agent-using-googles-gemini-api-with-intelligent-adaptation-features/>
35. Vibe Coding vs. Agentic Coding: Fundamentals and Practical Implications of Agentic AI - arXiv, accessed on July 10, 2025, <https://arxiv.org/pdf/2505.19443>
36. Hardest Monotone Functions for Evolutionary Algorithms - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2311.07438v2>
37. Monotonicity of fitness landscapes and mutation rate control, accessed on July 10, 2025, <https://repository.mdx.ac.uk/item/864q4>
38. cset.georgetown.edu, accessed on July 10, 2025,
<https://cset.georgetown.edu/article/how-to-improve-ai-red-teaming-challenges-and-recommendations/#::~:~:text=An%20AI%20red%2Dteaming%20exercise,to%20elicit%20the%20unwanted%20behaviors.>

39. LLM red teaming guide (open source) | Promptfoo, accessed on July 10, 2025, <https://www.promptfoo.dev/docs/red-team/>
40. Red Teaming for Large Language Models: A Comprehensive Guide - Coralogix, accessed on July 10, 2025, <https://coralogix.com/ai-blog/red-teaming-for-large-language-models-a-comprehensive-guide/>
41. Building Safe GenAI Applications: An End-to-End Overview of Red Teaming for Large Language Models - ACL Anthology, accessed on July 10, 2025, <https://aclanthology.org/2025.trustnlp-main.23.pdf>
42. AI Red Teaming 101 – Full Course (Episodes 1-10) - YouTube, accessed on July 10, 2025, <https://www.youtube.com/watch?v=DwFVhFdD2fs>
43. AART: AI-Assisted Red-Teaming with Diverse Data Generation for New LLM-powered Applications - ACL Anthology, accessed on July 10, 2025, <https://aclanthology.org/2023.emnlp-industry.37/>
44. Advancing AI Research at NAACL 2025 - Capital One, accessed on July 10, 2025, <https://www.capitalone.com/tech/ai/advancing-ai-research-naacl-2025/>
45. AI Agent Architecture: Breaking Down the Framework of Autonomous Systems - Kanerika, accessed on July 10, 2025, <https://kanerika.com/blogs/ai-agent-architecture/>
46. AI Agents: Evolution, Architecture, and Real-World Applications - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2503.12687v1>
47. A practical guide to the architectures of agentic applications - Speakeasy, accessed on July 10, 2025, <https://www.speakeasy.com/mcp/ai-agents/architecture-patterns>
48. Unveiling the Layers of Agentic AI | by Cobus Greyling | Jun, 2025 - Medium, accessed on July 10, 2025, <https://cobusgreyling.medium.com/unveiling-the-layers-of-agentic-ai-84c5e889da08>
49. Agentic AI Needs a Systems Theory - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2503.00237v1>
50. AI Agent Architecture: Core Principles & Tools in 2025 | Generative AI Collaboration Platform, accessed on July 10, 2025, <https://orq.ai/blog/ai-agent-architecture>
51. Agentic AI Architectures And Design Patterns | by Anil Jain | AI / ML ..., accessed on July 10, 2025, <https://medium.com/@anil.jain.baba/agentic-ai-architectures-and-design-patterns-288ac589179a>
52. Agentic Design Patterns. From reflection to collaboration... | by Bijit Ghosh - Medium, accessed on July 10, 2025, <https://medium.com/@bijit211987/agentic-design-patterns-cbd0aae2962f>
53. SAGA: A Security Architecture for Governing AI Agentic ... - arXiv, accessed on July 10, 2025, <https://arxiv.org/pdf/2504.21034>
54. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation, accessed on July 10, 2025, <https://opt-ml.org/papers/2023/paper21.pdf>
55. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation |

- OpenReview, accessed on July 10, 2025, <https://openreview.net/forum?id=46Zgqo4QIU>
56. arXiv:2310.02304v3 [cs.CL] 16 Aug 2024, accessed on July 10, 2025, <https://arxiv.org/pdf/2310.02304>
 57. Large language model based mutations in genetic improvement - PMC, accessed on July 10, 2025, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11750896/>
 58. integration of large language models and evolutionary ... - arXiv, accessed on July 10, 2025, <https://arxiv.org/pdf/2505.05756>
 59. codelion/openevolve: Open-source implementation of AlphaEvolve - GitHub, accessed on July 10, 2025, <https://github.com/codelion/openevolve>
 60. Evolving code with a large language model - ResearchGate, accessed on July 10, 2025, https://www.researchgate.net/publication/383984353_Evolving_code_with_a_large_language_model
 61. LLM-Meta-SR: Learning to Evolve Selection Operators for Symbolic Regression - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2505.18602v1>
 62. [2401.07102] Evolving Code with A Large Language Model - arXiv, accessed on July 10, 2025, <https://arxiv.org/abs/2401.07102>
 63. How to Teach an LLM Without Buying It — Part 2: Evolutionary Algorithms | by David Alami, accessed on July 10, 2025, <https://davidalami.medium.com/how-to-teach-an-llm-without-buying-it-part-2-evolutionary-algorithms-791761a60bda?source=rss-----ai-5>
 64. What is Continual Learning? - IBM, accessed on July 10, 2025, <https://www.ibm.com/think/topics/continual-learning>
 65. LifelongAgentBench: A Benchmark for Evaluating Continuous Learning in LLM-Based Agents - MarkTechPost, accessed on July 10, 2025, <https://www.marktechpost.com/2025/06/04/lifelongagentbench-a-benchmark-for-evaluating-continuous-learning-in-llm-based-agents/>
 66. [2503.04655] CLDyB: Towards Dynamic Benchmarking for Continual Learning with Pre-trained Models - arXiv, accessed on July 10, 2025, <https://www.arxiv.org/abs/2503.04655>
 67. CLDyB: Towards Dynamic Benchmarking for Continual Learning with Pre-trained Models, accessed on July 10, 2025, <https://openreview.net/forum?id=RnxwxGXxex>
 68. Towards Large Language Models as Copilots for Theorem Proving in Lean - Math-AI, accessed on July 10, 2025, <https://mathai2023.github.io/papers/4.pdf>
 69. Benchmarking Automated Theorem Proving with Large Language Models - ACL Anthology, accessed on July 10, 2025, <https://aclanthology.org/2024.nlp4science-1.18.pdf>
 70. LeanDojo: Theorem Proving with Retrieval-Augmented Language ..., accessed on July 10, 2025, <https://leandojo.org/>
 71. LeanAgent: Lifelong Learning for Formal Theorem Proving - OpenReview, accessed on July 10, 2025, <https://openreview.net/forum?id=Uo4EHT4ZZ8>
 72. [2410.06209] LeanAgent: Lifelong Learning for Formal Theorem Proving - arXiv, accessed on July 10, 2025, <https://arxiv.org/abs/2410.06209>

73. Synthetic Theorem Generation in Lean | OpenReview, accessed on July 10, 2025, <https://openreview.net/forum?id=EeDSMy5Ruj>
74. Large Language Models as Copilots for Theorem Proving in Lean - arXiv, accessed on July 10, 2025, <https://arxiv.org/html/2404.12534v2>
75. Collaborative Theorem Proving with Large Language Models: Enhancing Formal Proofs with ProofRefiner | OpenReview, accessed on July 10, 2025, <https://openreview.net/forum?id=y9xNQZjUJM>
76. Large Language Models as Copilots for Theorem Proving in Lean - arXiv, accessed on July 10, 2025, <https://arxiv.org/abs/2404.12534>
77. How Math Reveals the Bleeding Edge of AI | Feature | PNNL, accessed on July 10, 2025, <https://www.pnnl.gov/news-media/how-math-reveals-bleeding-edge-ai>
78. The Symbol Grounding Problem - University of Oxford Department of Computer Science, accessed on July 10, 2025, https://www.cs.ox.ac.uk/activities/ieg/e-library/sources/harnad90_sgproblem.pdf
79. [2412.16075] Formal Mathematical Reasoning: A New Frontier in AI - arXiv, accessed on July 10, 2025, <https://arxiv.org/abs/2412.16075>
80. 23 Mathematical challenges, accessed on July 10, 2025, <https://lms.ac.uk/23-mathematical-challenges/>
81. AI is Helping Tackle One of The Biggest Unsolved Problems In Maths, accessed on July 10, 2025, <https://international-maths-challenge.com/ai-is-helping-tackle-one-of-the-biggest-unsolved-problems-in-maths/>
82. SQ3. What are the most inspiring open grand challenge problems? | One Hundred Year Study on Artificial Intelligence (AI100) - Stanford University, accessed on July 10, 2025, <https://ai100.stanford.edu/gathering-strength-gathering-storms-one-hundred-year-study-artificial-intelligence-ai100-2021-1/sq3>
83. The Future of Materials Science: AI, Automation, and Policy Strategies | Mercatus Center, accessed on July 10, 2025, <https://www.mercatus.org/research/policy-briefs/future-materials-science-ai-automation-and-policy-strategies>
84. AI-Driven Drug Discovery: A Comprehensive Review | ACS Omega, accessed on July 10, 2025, <https://pubs.acs.org/doi/10.1021/acsomega.5c00549>
85. AI for Scientific Discovery - IBM Research, accessed on July 10, 2025, <https://research.ibm.com/projects/ai-for-scientific-discovery>
86. Materials Science Meets AI - Number Analytics, accessed on July 10, 2025, <https://www.numberanalytics.com/blog/materials-science-meets-ai>
87. I made a CLI for improving prompts using a genetic algorithm : r/LocalLLaMA - Reddit, accessed on July 10, 2025, https://www.reddit.com/r/LocalLLaMA/comments/1hvayr2/i_made_a_cli_for_improving_prompts_using_a/
88. Big Ideas Lab unlocks the secrets of drug discovery through supercomputing and AI, accessed on July 10, 2025, <https://www.llnl.gov/article/52516/big-ideas-lab-unlocks-secrets-drug-discovery-through-supercomputing-ai>

89. Nanome: Virtual Reality for Drug Design and Molecular Visualization, accessed on July 10, 2025, <https://nanome.ai/>
90. AIDDISON™ AI Drug Discovery - Sigma-Aldrich, accessed on July 10, 2025, <https://www.sigmaaldrich.com/US/en/services/software-and-digital-platforms/aid-dison-ai-powered-drug-discovery>
91. Top Drug Discovery Software Solutions to Watch in 2025 | deepmirror, accessed on July 10, 2025, <https://www.deepmirror.ai/post/top-drug-discovery-software-solutions-to-watch-in-2025>
92. AI-Driven Drug Design (AIDD) - Simulations Plus, accessed on July 10, 2025, <https://www.simulations-plus.com/software/admetpredictor/ai-driven-drug-design-aid-d/>
93. AI Driven Drug Discovery: 5 Powerful Breakthroughs in 2025 - Lifebit, accessed on July 10, 2025, <https://lifebit.ai/blog/ai-driven-drug-discovery/>
94. Top Scientific Data Platforms for AI-Driven Drug Discovery: What to Look For - CDD Vault, accessed on July 10, 2025, <https://www.collaborativedrug.com/cdd-blog/top-scientific-data-platforms-for-ai-driven-drug-discovery-what-to-look-for>
95. Artificial Intelligence for Materials Science - mpie.de, accessed on July 10, 2025, https://www.mpie.de/4867299/artificial_intelligence
96. Machine Learning Speeds up Simulations in Material Science - KIT, accessed on July 10, 2025, https://www.kit.edu/kit/english/pi_2021_049_machine-learning-speeds-up-simulations-in-material-science.php
97. www.devoteam.com, accessed on July 10, 2025, <https://www.devoteam.com/expert-view/human-in-the-loop-what-how-and-why/#:~:text=In%20the%20human%2Din%2Dthe,AI%20model%20throughout%20the%20process.>
98. Why AI still needs you: Exploring Human-in-the-Loop systems ..., accessed on July 10, 2025, <https://workos.com/blog/why-ai-still-needs-you-exploring-human-in-the-loop-systems>
99. Agents with Human in the Loop : Everything You Need to Know - DEV Community, accessed on July 10, 2025, <https://dev.to/camelai/agents-with-human-in-the-loop-everything-you-need-to-know-3fo5>
100. Human-In-The-Loop: What, How and Why | Devoteam, accessed on July 10, 2025, <https://www.devoteam.com/expert-view/human-in-the-loop-what-how-and-why/>
101. Human-in-the-Loop for AI Agents: Best Practices, Frameworks, Use Cases, and Demo, accessed on July 10, 2025, <https://www.permit.io/blog/human-in-the-loop-for-ai-agents-best-practices-frameworks-use-cases-and-demo>
102. What is chain of thought (CoT) prompting? - IBM, accessed on July 10, 2025,

<https://www.ibm.com/think/topics/chain-of-thoughts>

103. Advancing AI Reasoning: An Intro From StaR to DeepSeek | by Joe El Khoury - Medium, accessed on July 10, 2025,
<https://medium.com/@jelkhoury880/advancing-ai-reasoning-a-comprehensive-report-4982b7c19bdc>
104. Master Chain-of-Thought Prompting Techniques for AI - Relevance AI, accessed on July 10, 2025,
<https://relevanceai.com/prompt-engineering/master-chain-of-thought-prompting-techniques-for-ai>
105. AI-Powered Workplace Reasoning: Unlocking the Potential of Chain of Thought in 2025, accessed on July 10, 2025,
<https://www.dpexnetwork.org/articles/ai-powered-workplace-reasoning-unlocking-the-potential-of-chain-of-thought-in-2025>